LAPORAN PRAKTIKUM UJIAN TENGAH SEMESTER (UTS)
MATA KULIAH PEMROGRAMAN MULTIPLATFORM
(KELAS A)

"PENGEMBANGAN APLIKASI FINTRACK - CATATAN KEUANGAN PRIBADI BERBASIS FLUTTER DENGAN PENERAPAN ASYNCHRONOUS PROGRAMMING, SQLITE, DAN SHARED PREFERENCES"



Disusun Oleh:
NOVAL DWI PAMUNGKAS
NIM. 240605110040
Kelas: A (Semester 5)

Dosen Pengampu:
A'la Syauqi, M.Kom

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM

MALANG
2026

BAB I
PENDAHULUAN

1.1. Latar Belakang dan Gambaran Aplikasi

Pengelolaan keuangan pribadi merupakan aspek yang sangat krusial dalam kehidupan modern, khususnya bagi kalangan mahasiswa dan masyarakat produktif. Kurangnya disiplin serta rendahnya visibilitas terhadap arus keluar dan masuknya uang harian kerap kali menimbulkan perilaku konsumtif yang berujung pada defisit anggaran. Metode konvensional seperti pencatatan di buku kas fisik atau aplikasi spreadsheet seringkali ditinggalkan karena dinilai tidak praktis, lambat, dan rentan hilang saat mobilitas tinggi.

Perkembangan pesat teknologi perangkat bergerak multiplatform memungkinkan perancangan aplikasi yang dapat beroperasi secara konsisten lintas sistem operasi (Android, iOS, dan Desktop) menggunakan basis kode tunggal (*single codebase*). Menggunakan framework Flutter dan bahasa pemrograman Dart, aplikasi **FinTrack (Financial Tracker)** dikembangkan guna memberikan solusi cerdas, cepat, dan terstruktur dalam mendokumentasikan setiap transaksi pemasukan maupun pengeluaran.

FinTrack mengusung filosofi desain *offline-first*, memastikan seluruh pencatatan data dan komputasi saldo dapat berjalan mandiri tanpa mewajibkan ketersediaan koneksi internet. Sasaran pengguna aplikasi ini adalah mahasiswa, pekerja lepas, profesional muda, dan masyarakat luas yang menginginkan rekapitulasi arus kas harian secara instan. Keunggulan utama aplikasi FinTrack terletak pada integrasi konsep *asynchronous programming* yang menjamin antarmuka tetap responsif tanpa hambatan (non-blocking UI), serta perpaduan dua teknik persistensi data lokal, yaitu SQLite (untuk data relasional terstruktur) dan SharedPreferences (untuk penyimpanan preferensi konfigurasi pengguna).

1.2. Fitur Aplikasi

Guna memenuhi kebutuhan pengelolaan arus kas harian secara optimal, aplikasi FinTrack menyediakan fitur-fitur utama sebagai berikut:

- **Dashboard Arus Kas Dinamis (Summary Card):** Menampilkan ringkasan total saldo bersih saat ini, total akumulasi pemasukan, dan total pengeluaran secara real-time. Dilengkapi badge status surplus/defisit dan progress bar visual pemakaian anggaran bulanan.

- **Manajemen Transaksi Penuh (CRUD SQLite):** Mendukung penambahan catatan transaksi baru (Create), penampilan riwayat transaksi dengan pengurutan kronologis terbaru (Read), pengubahan catatan (Update), dan penghapusan data transaksi melalui gesture Swipe-to-Dismiss yang dilengkapi dialog konfirmasi (Delete).
- **Pencarian Real-Time dan Filter Kategori Cepat:** Memungkinkan pencarian instan berdasarkan kata kunci judul maupun kategori transaksi, serta tombol filter cepat (Semua Transaksi, Pemasukan, Pengeluaran).
- **Visualisasi Statistik Kategori:** Menyajikan persentase proporsi alokasi pengeluaran dan pemasukan per kategori menggunakan visualisasi progress bar yang sederhana namun sangat informatif.
- **Banner Edukasi Finansial Dinamis (Simulasi Asinkron Jaringan):** Menampilkan kutipan dan tips bijak finansial yang dimuat secara asinkron dengan simulasi latensi jaringan (network delay), dilengkapi tombol refresh interaktif.
- **Preferensi Pengguna Persisten (SharedPreferences):** Pengaturan tema aplikasi (Dark Mode / Light Mode) yang tersimpan permanen di memori lokal, kustomisasi nama pengguna untuk salam personal di header dashboard, serta penentuan batas anggaran bulanan.
- **Manajemen dan Reset Basis Data:** Fasilitas pengosongan seluruh riwayat transaksi pada basis data SQLite dengan sistem proteksi dialog konfirmasi.

BAB II
PEMBAHASAN

2.1. Desain Antarmuka Pengguna (User Interface)

Desain antarmuka aplikasi FinTrack mengadopsi kaidah Material Design 3 yang mengedepankan estetika bersih, keterbacaan tinggi, dan kenyamanan visual. Skema warna utama menggunakan warna biru elegan (#2563EB) pada mode terang dan slate/navy (#0B132B / #1C2541) pada mode gelap. Setiap halaman dirancang dengan hierarki visual yang jelas untuk memudahkan pengguna memahami kondisi keuangan mereka secara sekilas.



Gambar 2.1 Tampilan Dashboard Utama (Mode Terang / Light Mode)

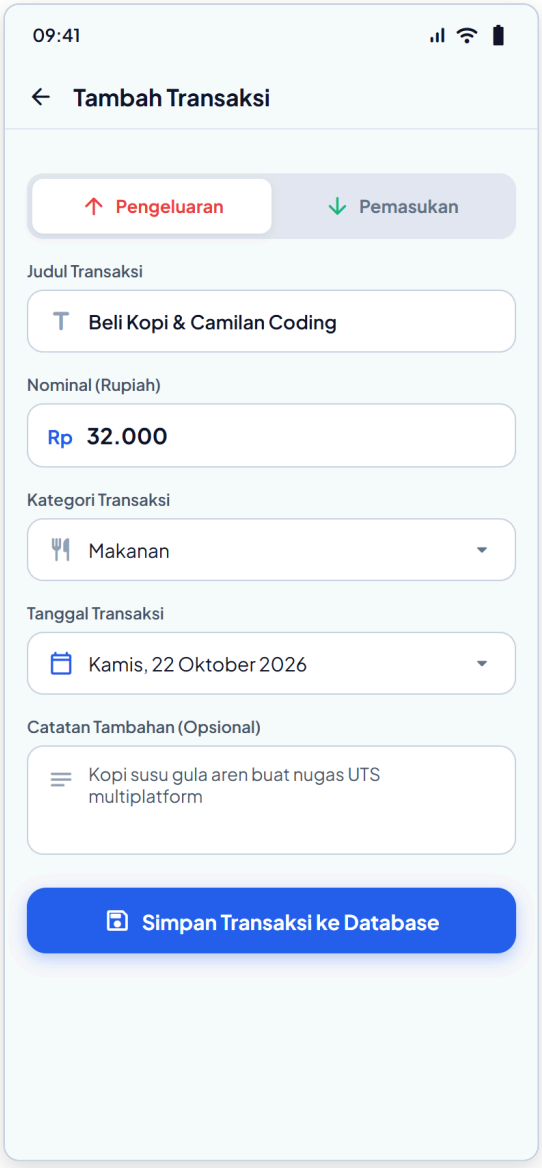
Gambar 2.1 memperlihatkan antarmuka halaman utama FinTrack pada mode terang. Bagian atas memuat AppBar dengan avatar inisial dan nama pengguna ("Noval Dwi Pamungkas"), kartu gradien ringkasan saldo bersih dengan indikator status surplus, kolom pemasukan dan pengeluaran, serta bar persentase pemakaian anggaran bulanan. Di bawahnya terdapat banner kutipan finansial asinkron, kotak pencarian realtime, filter chip kategori, dan daftar kartu riwayat transaksi.



Gambar 2.2 Tampilan Dashboard Utama (Mode Gelap / Dark Mode)

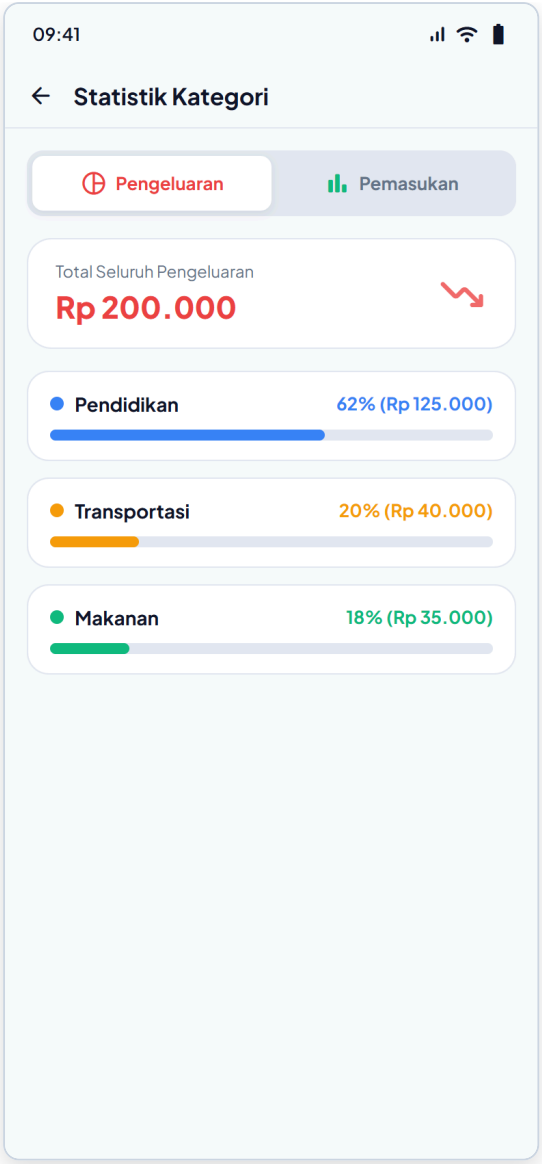
Gambar 2.2 menyajikan tampilan dashboard dalam mode gelap (Dark Mode). Fitur ini secara efektif mengurangi kelelahan mata saat penggunaan di malam hari serta menghemat konsumsi daya baterai. Status tema ini disimpan

secara persisten di SharedPreferences sehingga tidak kembali ke default saat aplikasi dibuka kembali.



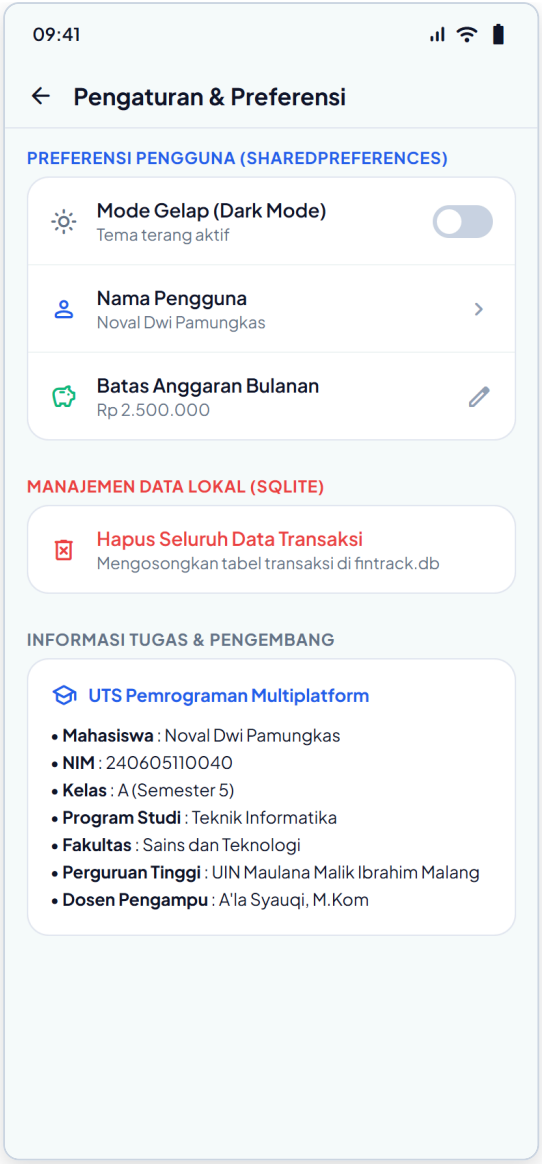
Gambar 2.3 Tampilan Form Tambah Transaksi Keuangan

Gambar 2.3 menampilkan formulir pencatatan transaksi baru. Pengguna dapat memilih jenis transaksi (Pengeluaran atau Pemasukan) melalui SegmentedButton, mengisi judul transaksi, memasukkan nominal angka dalam format Rupiah, memilih kategori dari dropdown, menentukan tanggal transaksi melalui dialog DatePicker interaktif, serta menambahkan catatan detail sebelum menyimpannya ke basis data SQLite.



Gambar 2.4 Tampilan Visualisasi Statistik Kategori

Gambar 2.4 menyajikan visualisasi statistik pengeluaran per kategori. Kartu ringkasan teratas menampilkan total akumulasi pengeluaran (Rp 200.000), diikuti oleh daftar persentase kontribusi masing-masing kategori (Pendidikan 62%, Transportasi 20%, Makanan 18%) yang dihitung melalui query agregat SQL.



Gambar 2.5 Tampilan Halaman Pengaturan dan Preferensi Pengguna

Gambar 2.5 memperlihatkan halaman pengaturan yang mengelola preferensi SharedPreferences (switch Dark Mode, dialog ubah nama pengguna, edit batas budget bulanan), manajemen data SQLite (opsi reset database), serta informasi identitas pengembang dan mata kuliah.

2.2. Persiapan Project dan Package

Pengembangan project aplikasi FinTrack dimulai dengan inisialisasi menggunakan Flutter CLI:

```
flutter create --org com.novaldwi --project-name fintrack_app
fintrack_app
```

Arsitektur direktori diorganisasikan secara rapi dan modular untuk memudahkan pemeliharaan kode program:

- `lib/models/` : Memuat `transaction_model.dart` untuk merepresentasikan objek data transaksi keuangan beserta metode serialisasi `toMap()` dan `fromMap()`.
- `lib/services/` : Memuat modul akses data lokal yaitu `database_helper.dart` (SQLite), `preference_service.dart` (SharedPreferences), dan `quote_service.dart` (simulasi asinkron).
- `lib/screens/` : Memuat antarmuka pengguna halaman beranda, formulir input transaksi, statistik visual, dan pengaturan.
- `lib/widgets/` : Memuat komponen antarmuka mandiri yang dapat digunakan kembali seperti `summary_card.dart` dan `transaction_card.dart`.
- `lib/main.dart` : Entry point utama yang menangani inisialisasi awal Flutter binding, engine FFI, dan konfigurasi tema.

Dependensi paket eksternal ditambahkan pada berkas `pubspec.yaml` sesuai dengan kebutuhan spesifikasi tugas UTS:

Nama Package	Versi	Fungsi dan Alasan Penggunaan
sqflite	^2.4.5	Menyediakan engine basis data SQLite lokal untuk Android dan iOS, memungkinkan eksekusi query relasional terstruktur (CRUD).
sqflite_common_ffi	^2.4.4	Dukungan binding FFI C SQLite untuk platform Desktop (Windows) serta automated unit test environment.
shared_preferences	^2.5.6	Menyimpan data preferensi pengguna dalam bentuk pasangan kunci dan nilai (key-value) pada memori lokal perangkat.
path_provider	^2.1.6	Mendeteksi lokasi path direktori sistem yang sah untuk penempatan berkas basis data SQLite pada perangkat pengguna.
path	^1.9.1	Utilitas manipulasi dan penggabungan path direktori secara aman lintas sistem operasi.
intl	^0.20.3	Memformat angka ke mata uang Rupiah Indonesia (Rp) dan memformat tanggal/waktu sesuai kaidah bahasa lokal (id_ID).

2.3. Implementasi Asynchronous Programming

Framework Flutter mengadopsi model eksekusi *single-threaded event loop*. Seluruh operasi I/O yang berhubungan dengan penyimpanan fisik (disk), database, atau layanan eksternal membutuhkan durasi waktu tertentu. Apabila operasi ini dijalankan secara sinkron (blocking), alur rendering antarmuka akan terhenti dan memicu fenomena *UI freeze* (aplikasi macet sementara).

Oleh karena itu, konsep *asynchronous programming* menggunakan konstruksi `Future`, `async`, dan `await` diterapkan di seluruh lapisan aplikasi. Pada berkas `lib/screens/home_screen.dart`, inisialisasi awal aplikasi memanfaatkan metode `Future.wait` untuk mengeksekusi tiga operasi I/O secara paralel:

```
// Memuat data preferensi, data SQLite, dan quote tips secara simultan
Future<void> _loadAllInitialData() async {
  setState(() => _isLoadingData = true); // Tampilkan indikator loading

  // Eksekusi 3 fungsi asinkron secara paralel tanpa saling menghambat
  await Future.wait([
    _loadPreferences(),   // Membaca SharedPreferences (Key-Value)
    _loadTransactions(),  // Query database SQLite (Structured)
    _loadDailyQuote(),    // Simulasi pengambilan kutipan finansial
eksternal
  ]);

  if (mounted) {
    setState(() => _isLoadingData = false); // Perbarui tampilan
antarmuka
  }
}
```

Dengan mekanisme ini, pengguna disajikan indikator loading (`CircularProgressIndicator`) saat operasi sedang berlangsung di latar belakang. Begitu seluruh Future terselesaikan, state aplikasi diperbarui dan data dirender secara mulus ke layar.

Selain itu, simulasi pengambilan data eksternal diterapkan pada berkas `lib/services/quote_service.dart` menggunakan `Future.delayed` untuk merepresentasikan latensi pemanggilan REST API:

```
// Simulasi pemanggilan API eksternal dengan latensi jaringan 800 ms
static Future<String> fetchDailyFinancialTip() async {
  await Future.delayed(const Duration(milliseconds: 800));

  final random = Random();
  final index = random.nextInt(_tips.length);
  return _tips[index]; // Mengembalikan kutipan finansial acak
}
```

2.4. Implementasi Data Persistence

Sesuai ketentuan tugas UTS, aplikasi FinTrack menerapkan dua teknik penyimpanan data lokal:

2.4.1. Teknik Pertama: Shared Preferences (Key-Value Storage)

Alasan Pemilihan: SharedPreferences sangat efisien dan ringan untuk menyimpan data status atau preferensi konfigurasi pengguna tanpa memerlukan beban pembuatan tabel relasional.

Jenis Data yang Disimpan:

1. `is_dark_mode` (tipe `bool`): Menyimpan pilihan tema gelap (true) atau terang (false).
2. `user_name` (tipe `String`): Menyimpan nama pengguna untuk personalisasi sapaan dashboard.
3. `monthly_budget` (tipe `double`): Menyimpan batas target anggaran belanja bulanan.

```
// Implementasi pada lib/services/preference_service.dart
class PreferenceService {
  static const String _keyDarkMode = 'is_dark_mode';
  static const String _keyUserName = 'user_name';
  static const String _keyMonthlyBudget = 'monthly_budget';

  // Membaca tema secara asinkron
  static Future<bool> isDarkMode() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(_keyDarkMode) ?? false;
  }

  // Menyimpan tema secara asinkron
  static Future<bool> setDarkMode(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    return await prefs.setBool(_keyDarkMode, value);
  }

  // Membaca nama pengguna
  static Future<String> getUserName() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_keyUserName) ?? 'Noval Dwi Pamungkas';
  }
}
```

2.4.2. Teknik Kedua: SQLite (Structured Database Storage)

Alasan Pemilihan: Data transaksi keuangan memiliki struktur relasional dengan banyak kolom serta membutuhkan kapabilitas pencarian teks (`LIKE`), filter tipe transaksi, pengurutan waktu kronologis (`ORDER BY date DESC`), dan perhitungan matematis total saldo (`SUM`).

Skema Tabel Transaksi:

```
CREATE TABLE transactions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  amount REAL NOT NULL,
  type TEXT NOT NULL,
  category TEXT NOT NULL,
  date TEXT NOT NULL,
  note TEXT
);
```

Operasi CRUD dan kalkulasi agregat diimplementasikan dalam berkas `lib/services/database_helper.dart`:

```
// CREATE: Menambah transaksi baru
Future<int> insertTransaction(TransactionModel transaction) async {
  final db = await database;
  return await db.insert('transactions', transaction.toMap());
}

// READ: Mengambil daftar transaksi dengan proteksi parameter SQL
Injection
Future<List<TransactionModel>> getAllTransactions({String? filterType,
String? searchKeyword}) async {
  final db = await database;
  List<String> conditions = [];
  List<dynamic> whereArgs = [];

  if (filterType != null && filterType != 'all') {
    conditions.add('type = ?');
    whereArgs.add(filterType);
  }
  if (searchKeyword != null && searchKeyword.trim().isNotEmpty) {
    conditions.add('(title LIKE ? OR category LIKE ? OR note LIKE ?)');
    final term = '%${searchKeyword.trim()}%';
    whereArgs.addAll([term, term, term]);
  }

  final result = await db.query(
    'transactions',
    where: conditions.isNotEmpty ? conditions.join(' AND ') : null,
    whereArgs: whereArgs.isNotEmpty ? whereArgs : null,
    orderBy: 'date DESC, id DESC',
  );
  return result.map((json) => TransactionModel.fromMap(json)).toList();
}

// READ AGGREGATE: Menghitung total saldo, pemasukan, dan pengeluaran
secara matematis
Future<Map<String, double>> getSummaryTotals() async {
  final db = await database;
  final incomeRes = await db.rawQuery("SELECT SUM(amount) as total FROM
transactions WHERE type = 'income'");
  final expenseRes = await db.rawQuery("SELECT SUM(amount) as total
FROM transactions WHERE type = 'expense'");

  final double income = (incomeRes.first['total'] as num?)?.toDouble()
?? 0.0;
  final double expense = (expenseRes.first['total'] as
num?)?.toDouble() ?? 0.0;
  return {'income': income, 'expense': expense, 'balance': income -
expense};
}

// DELETE: Menghapus catatan berdasarkan Primary Key
Future<int> deleteTransaction(int id) async {
  final db = await database;
  return await db.delete('transactions', where: 'id = ?', whereArgs:
```

```
[id]);
}
```

2.5. Pengujian dan Hasil Implementasi

Pengujian fungsionalitas dan ketahanan data dilakukan menggunakan automated test suite (`flutter test`) serta pengujian manual pada seluruh skenario kerja aplikasi. Hasil pengujian dirangkum pada tabel berikut:

No.	Fitur yang Diuji	Langkah Pengujian	Hasil yang Diharapkan	Hasil Pengujian
1	Fitur utama aplikasi (CRUD Transaksi)	Membuka form tambah transaksi, menginput data, menekan simpan.	Data berhasil tersimpan ke tabel SQLite dan langsung muncul di daftar riwayat.	**Berhasil**
2	Proses asinkron (Future, async, await)	Menjalankan pemanggilan data SQLite, SharedPreferences, dan quote tips melalui Future.wait.	Proses berjalan non-blocking di latar belakang tanpa freeze, indikator loading tampil saat memuat data.	**Berhasil**
3	Penyimpanan data pertama (SharedPreferences)	Mengubah tema ke Dark Mode dan mengubah nama pengguna di menu pengaturan.	Status tema dan nama pengguna berhasil tersimpan secara permanen dalam pasangan key-value.	**Berhasil**
4	Penyimpanan data kedua (SQLite)	Memasukkan beberapa transaksi pengeluaran dan pemasukan, kemudian melihat ringkasan saldo.	Data tersimpan terstruktur dalam tabel transactions, nilai agregat SUM dihitung dengan akurat.	**Berhasil**
5	Persistensi data (Offline & Reboot)	Menutup paksa aplikasi (kill process), memutuskan jaringan internet, lalu membuka kembali aplikasi.	Seluruh catatan transaksi SQLite dan preferensi SharedPreferences tetap tersedia secara utuh.	**Berhasil**
6	Visualisasi Statistik Kategori	Membuka halaman statistik kategori dari AppBar beranda.	Pengeluaran dikelompokkan (GROUP BY) dan ditampilkan dalam visual bar persentase yang akurat.	**Berhasil**

BAB III
PENUTUP

3.1. Kesimpulan

Berdasarkan perancangan, implementasi kode program, dan pengujian sistem yang telah dilaksanakan, diperoleh kesimpulan sebagai berikut:

- Aplikasi FinTrack berhasil dibangun menggunakan framework Flutter dan bahasa pemrograman Dart dengan menerapkan arsitektur kode yang bersih, terpisah per modul (models, services, screens, widgets), dan mematuhi kaidah Material Design 3.
- Konsep *Asynchronous Programming* menggunakan konstruksi `Future`, `async`, `await`, `Future.wait`, dan `Future.delayed` berhasil diimplementasikan secara optimal pada seluruh operasi I/O basis data, preferensi, dan pemanggilan data eksternal, sehingga antarmuka tetap responsif dan bebas dari hambatan (*jank*).
- Dua teknik persistensi data lokal diterapkan secara tepat guna: **SharedPreferences** untuk preferensi status aplikasi berpasangan key-value (mode tema, nama pengguna, target budget), dan **SQLite** untuk penyimpanan data transaksional relasional terstruktur (CRUD transaksi dan perhitungan agregat saldo).
- Pengujian persistensi membuktikan bahwa aplikasi mampu beroperasi secara *offline-first* dengan integritas data yang terjaga sempurna setelah aplikasi ditutup paksa maupun dimulai ulang.

3.2. Saran

Untuk pengembangan aplikasi FinTrack pada tahap selanjutnya, beberapa rekomendasi yang dapat diterapkan antara lain:

- Menambahkan integrasi sinkronisasi data berbasis Cloud (seperti Firebase Cloud Firestore atau Supabase) agar riwayat transaksi dapat diakses dari beberapa perangkat secara bersamaan (*multi-device sync*).
- Mengimplementasikan fitur ekspor dan impor laporan keuangan ke format berkas Excel (.xlsx) atau dokumen PDF menggunakan `path_provider` dan package cetak lokal.
- Menambahkan visualisasi grafik diagram lingkaran (Pie Chart) interaktif menggunakan pustaka `fl_chart` untuk analisis alokasi anggaran yang lebih komprehensif.

- Menerapkan sistem autentikasi biometrik (Fingerprint / Face ID) menggunakan package `local_auth` guna memperkuat privasi data keuangan pengguna.

DAFTAR PUSTAKA

Flutter Documentation. (2026). *Asynchronous Programming: Futures and Async-Await in Dart*. Google Developers. Diakses dari https://docs.flutter.dev/codelabs/async-await

Flutter Documentation. (2026). *Persist data with SQLite*. Google Developers. Diakses dari https://docs.flutter.dev/cookbook/persistence/sqlite

Flutter Documentation. (2026). *Store key-value data on disk with shared_preferences*. Google Developers. Diakses dari https://docs.flutter.dev/cookbook/persistence/key-value

Material Design. (2026). *Material 3 Design Guidelines*. Google. Diakses dari https://m3.material.io/

Tekse, A. (2025). *sqflite: SQLite plugin for Flutter*. Pub.dev Repository. Diakses dari https://pub.dev/packages/sqflite

The Dart Project. (2026). *intl: Internationalization and localization support for Dart*. Pub.dev Repository. Diakses dari https://pub.dev/packages/intl